

Network Digital Twin

Complete Implementation Guide

3-Tier Hierarchical Architecture on GNS3

From Network Configuration to Live Digital Twin

Overview

This guide walks you through every step required to transform your configured GNS3 3-tier network (Core, Distribution, Access layers) into a fully operational Network Digital Twin — a virtual, real-time replica of your network used for monitoring, simulation, testing, and AI-driven prediction.

Architecture: 2 Core Switches | 4 Distribution Switches | 4 Access Switches | 4 End Hosts

What you already have: GNS3 topology configured with IP addressing, VLANs, routing protocols (OSPF/EIGRP), STP, HSRP, and port security.

Phase 1 — Export & Document Your GNS3 Network

Step 1: Export Running Configurations

From each device in GNS3, export the full running configuration. Open the GNS3 console for each device and run:

```
show running-config
```

Save each device's output in a structured folder:

```
/configs/core/core-sw1.txt  
/configs/core/core-sw2.txt  
/configs/distribution/dist-sw1.txt ... dist-sw4.txt  
/configs/access/acc-sw1.txt ... acc-sw4.txt
```

Step 2: Capture Network State

On each device, run the following commands and save their output:

- show ip route — routing table snapshot
- show interfaces — interface states and counters
- show vlan brief — VLAN assignments
- show spanning-tree — STP topology per VLAN
- show standby — HSRP active/standby states
- show ip ospf neighbor OR show ip eigrp neighbors — adjacency tables

Step 3: Build an IP & VLAN Inventory

Create a spreadsheet (or CSV) documenting every interface with its IP address, subnet mask, VLAN, role, and connected neighbor. This becomes the foundation of your Source of Truth (SoT).

Phase 2 — Build the Source of Truth (NetBox)

Step 4: Install NetBox

NetBox is the industry-standard open-source tool for documenting IP address space, devices, VLANs, and cabling. Install it on a dedicated server or VM:

```
# Option A: Docker (recommended for lab)
git clone https://github.com/netbox-community/netbox-docker.git
cd netbox-docker && docker-compose up -d
# Access at http://localhost:8080 (default user: admin / admin)
```

Step 5: Populate NetBox with Your Topology

Manually or via script, add all devices and data to NetBox:

1. Create Sites: e.g., 'GNS3 Lab'
2. Create Device Roles: Core Switch, Distribution Switch, Access Switch
3. Create Device Types: e.g., Cisco Catalyst 3850
4. Add all 10 Devices with their roles and sites
5. Add Interfaces to each device
6. Assign IP Addresses to every interface
7. Create VLANs and assign them to interfaces
8. Add Cables between devices to mirror your GNS3 topology

Tip: Use the NetBox REST API or netutils Python library to import your configs automatically instead of entering data manually.

Phase 3 — Enable Telemetry & Data Collection

Step 6: Enable SNMP on All GNS3 Devices

SSH into each GNS3 device and configure SNMPv2 or SNMPv3:

```
! Cisco IOS – SNMPv2c example
snmp-server community public RO
snmp-server location GNS3-Lab
snmp-server contact admin@lab.local
```

For SNMPv3 (more secure):

```
snmp-server group MONITOR v3 priv
snmp-server user SNMPuser MONITOR v3 auth sha Auth@123 priv aes 128 Priv@123
```

Step 7: Install the Telemetry Stack (TIG Stack)

Deploy Telegraf (collector) + InfluxDB (time-series DB) + Grafana (dashboard) using Docker Compose:

```
# docker-compose.yml
```

```
version: '3'
services:
  influxdb:
    image: influxdb:2.7
    ports: ['8086:8086']
  telegraf:
    image: telegraf:1.29
    volumes: ['./telegraf.conf:/etc/telegraf/telegraf.conf']
  grafana:
    image: grafana/grafana:10.0.0
    ports: ['3000:3000']
```

Step 8: Configure Telegraf to Poll Your Devices

Edit telegraf.conf to point at your GNS3 device management IPs:

```
[[inputs.snmp]]
  agents = ['udp://192.168.1.1:161', 'udp://192.168.1.2:161']
  community = 'public'
[[inputs.snmp.field]]
  name = 'cpu'
  oid = 'HOST-RESOURCES-MIB::hrProcessorLoad.1'
[[inputs.snmp.field]]
  name = 'uptime'
  oid = 'RFC1213-MIB::sysUpTime.0'
```

Telegraf will poll every device at configurable intervals (e.g., 60 seconds) and write metrics to InfluxDB.

Phase 4 — Automate Config Sync (Ansible)

Step 9: Install Ansible & Network Collections

```
pip install ansible
ansible-galaxy collection install cisco.ios
ansible-galaxy collection install ansible.netcommon
```

Step 10: Create Your Inventory File

Define all GNS3 devices in an Ansible inventory:

```
# inventory.yml
all:
  children:
    core:
      hosts:
        core-sw1: {ansible_host: 192.168.1.1}
        core-sw2: {ansible_host: 192.168.1.2}
    distribution:
```

```
hosts:
  dist-sw1: {ansible_host: 192.168.1.3}
  dist-sw2: {ansible_host: 192.168.1.4}
  dist-sw3: {ansible_host: 192.168.1.5}
  dist-sw4: {ansible_host: 192.168.1.6}
```

Step 11: Write a Playbook to Backup Configs

```
# backup_configs.yml
- name: Backup all device configs
  hosts: all
  gather_facts: no
  tasks:
    - name: Get running config
      cisco.ios.ios_command:
        commands: show running-config
      register: config_output
    - name: Save to file
      copy:
        content: '{{ config_output.stdout[0] }}'
        dest: './backups/{{ inventory_hostname }}.txt'
```

Schedule this playbook via cron to auto-sync configs from your live GNS3 twin to NetBox.

Phase 5 — Configuration Verification (Batfish)

Step 12: Install Batfish

Batfish analyzes network configurations without running traffic, allowing you to verify correctness before applying changes.

```
# Run Batfish server via Docker
docker pull batfish/allinone
docker run -d -p 9997:9997 -p 9996:9996 batfish/allinone
```

```
# Install Python client
pip install pybatfish
```

Step 13: Load Your Configs into Batfish

```
from pybatfish.client.session import Session
from pybatfish.datamodel import *

bf = Session(host='localhost')
bf.set_network('3tier-network')
bf.init_snapshot('./configs', name='baseline', overwrite=True)

# Check routing tables
```

```
bf.q.routes().answer().frame()
```

```
# Find unreachable interfaces
bf.q.interfaceProperties().answer().frame()
```

Use Batfish to test what-if scenarios: simulate a Core switch failure, add a new VLAN, or change OSPF costs — all before touching the real network.

Phase 6 — Live Dashboard (Grafana)

Step 14: Connect Grafana to InfluxDB

- Open Grafana at <http://localhost:3000>
- Go to Configuration > Data Sources > Add data source
- Select InfluxDB, enter URL: <http://influxdb:8086>
- Enter your bucket name and access token
- Click Save & Test

Step 15: Build Your Network Dashboard

Create panels for each key metric:

- Interface Utilization — line chart per interface
- CPU Usage — gauge per device (Core, Distribution, Access)
- Link States — status panel (Up/Down per link)
- OSPF/EIGRP Neighbor Count — stat panel
- HSRP Active/Standby State — status per VLAN
- Error Counters — table with top interfaces by errors

Pro Tip: Use Grafana Alerting to send email/Slack notifications when a link goes down or CPU exceeds 80%.

Phase 7 — Topology Visualization

Step 16: Use NetBox Topology Plugin or draw.io

Install the netbox-topology-views plugin to auto-generate a visual map from your NetBox data:

```
pip install netbox-topology-views
```

Or export your topology from NetBox via the REST API and render it using Python with the NetworkX + Matplotlib libraries:

```
import networkx as nx
import matplotlib.pyplot as plt
```

```
G = nx.Graph()
```

```
G.add_edge('Core-SW1', 'Dist-SW1')
G.add_edge('Core-SW1', 'Dist-SW2')
# ... add all edges
nx.draw(G, with_labels=True, node_color='lightblue')
plt.show()
```

Phase 8 — AI/ML Integration (Advanced)

Step 17: Collect Historical Telemetry Data

Allow your TIG stack to collect at least 2–4 weeks of metrics. Export data from InfluxDB to CSV for ML training:

```
# Query InfluxDB for historical CPU data
influx query 'from(bucket:"network") |> range(start: -30d) |> filter(fn: (r) =>
r._measurement == "cpu")' --raw > cpu_data.csv
```

Step 18: Train Anomaly Detection Models

Use Python with scikit-learn or TensorFlow to detect anomalies in traffic patterns:

```
from sklearn.ensemble import IsolationForest
import pandas as pd

df = pd.read_csv('cpu_data.csv')
model = IsolationForest(contamination=0.05)
df['anomaly'] = model.fit_predict(df[['cpu_usage']])
# anomaly == -1 means abnormal behavior
```

Step 19: Predictive Failure Detection

Use LSTM (Long Short-Term Memory) networks to predict future interface errors or congestion:

- Collect 30+ days of interface error counters
- Train LSTM on the time-series data
- Deploy model to predict failures 15-60 minutes in advance
- Trigger Grafana alerts before actual failure occurs

Tool Stack Summary

Tool / Platform	Purpose	Cost
GNS3	Network emulation (base)	Free
NetBox	Source of Truth / IPAM	Free (OSS)
Ansible	Config automation & sync	Free

Batfish	Config verification / what-if	Free
Telegraf	Metrics collection (SNMP)	Free
InfluxDB	Time-series database	Free (OSS)
Grafana	Dashboard & alerting	Free (OSS)
Python / Nornir	Custom scripting & automation	Free
scikit-learn / TF	ML anomaly detection	Free
Cisco CML	Enterprise-grade emulation	Paid

Implementation Checklist

Phase 1: Export configs, capture network state, build IP/VLAN inventory

Phase 2: Install NetBox, populate all devices, interfaces, IPs, VLANs, and cables

Phase 3: Enable SNMP on GNS3 devices, deploy TIG stack, configure Telegraf polling

Phase 4: Install Ansible, create inventory, automate config backup & sync

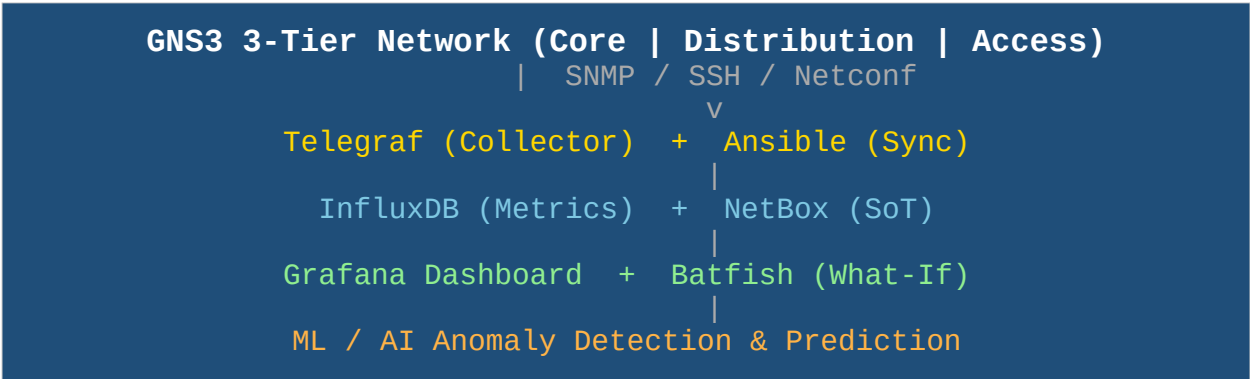
Phase 5: Install Batfish, load configs, run what-if analysis

Phase 6: Connect Grafana to InfluxDB, build live monitoring dashboard

Phase 7: Deploy topology visualization plugin or custom Python graph

Phase 8: Collect historical data, train ML models, deploy predictive alerts

Digital Twin Architecture Flow



End of Document